

Tech Stack & Architecture Documentation

Project: Tabular RAG Assistant (Heart Disease Dataset)

1. Architectural Overview

This application uses a decoupled client-server architecture. The frontend handles the heavy lifting for the 3D spatial UI and interactive data visualizations, while the Python backend processes the CSV, manages the vector database, and orchestrates the Ollama LLM interactions locally.

2. Frontend Layer (3D UI & Interaction)

To achieve the "Landmark Signage" aesthetic with floating 3D glassmorphic cards and spatial backgrounds, we need a modern, component-driven frontend.

- **Framework: React.js** (via Vite or Next.js).
 - *Why:* Provides the component architecture needed to manage the complex states of the chat interface and the 3D data gallery.
- **Styling & 3D CSS: Tailwind CSS.**
 - *Why:* Enables rapid implementation of the specific hex codes (#0A0C10 , #13161D , #00E5FF) and handles 3D CSS transforms (perspective , rotateX , rotateY , translateZ) through custom utility classes.
- **Animations: Framer Motion.**
 - *Why:* Essential for animating the 3D Patient Record cards sliding in from the Z-axis and managing the smooth layout shifts between PC and Mobile states.
- **Abstract 3D Canvas (Optional): React Three Fiber / Three.js.**
 - *Why:* Used specifically to render the slow-rotating 3D geometric wireframe in the background, matching the spatial aesthetic of the design reference.

3. Backend & Orchestration Layer (Python)

The backend manages data ingestion, similarity thresholding, and API communication.

- **Language: Python 3.10+**
- **API Framework: FastAPI.**
 - *Why:* Extremely fast, lightweight, and supports asynchronous requests. Perfect for streaming Ollama's responses back to the frontend chunk-by-chunk.

- **AI Orchestration: LangChain.**
 - *Why:* The core engine of the app. It will be used to load the CSV, apply the custom Semantic Row Conversion logic (turning CSV rows into sentences), and chain the retrieval results to Ollama.
- **Data Processing: Pandas** (Optional but recommended).
 - *Why:* For cleaning the `heart_dataset.csv` or handling missing values before passing the data to LangChain's CSVLoader.

4. Vector Storage Layer

- **Vector Database: Chroma (ChromaDB).**
 - *Why:* Mandatory for the project. Runs 100% locally. It natively calculates the Cosine Distance needed for the strict ">0.5 threshold" guardrail, ensuring out-of-context questions are blocked.

5. AI / Machine Learning Layer (100% Free & Local)

This layer operates entirely on your local machine, ensuring no data leaves your hardware and incurring \$0 API costs.

- **Embedding Model (Encoder):** `sentence-transformers/all-MiniLM-L6-v2` .
 - *Source:* Hugging Face (via LangChain's `HuggingFaceEmbeddings`).
 - *Why:* Extremely fast CPU performance. It will convert the semantic sentences (e.g., "Patient is a 40 year old Male with 289 Cholesterol...") into dense vectors for Chroma.
- **Regressive Model (LLM):** **Ollama** running `llama3` (8B) or `mistral` (7B).
 - *Why:* Mandatory for the project. Ollama provides a local, highly capable LLM server. Llama 3 is particularly good at following strict system prompts (like "Reply exactly with: 'Sorry I can't find anything relevant...'").

6. System Flow Pipeline

1. **Startup (Data Ingestion):**
 - FastAPI boots up. LangChain reads `heart_dataset.csv` .
 - Each row is parsed into a semantic string and vectorized using `all-MiniLM-L6-v2` .
 - Vectors and raw metadata are persisted into ChromaDB.
2. **User Query:**
 - React frontend sends a query via POST request to FastAPI.
3. **Retrieval & Guardrail Check:**

- Backend vectorizes the query and performs a similarity search in Chroma.
- *Threshold Check*: If the Cosine Distance > 0.5 , FastAPI immediately returns the exact failure string to the frontend.

4. **LLM Generation:**

- If the threshold passes, the semantic context and the query are passed to the Ollama API locally.
- Ollama processes the prompt at `Temperature = 0.0` to prevent hallucinations.

5. **Response Delivery:**

- FastAPI returns the AI's answer *and* the raw metadata of the retrieved rows.
- React frontend renders the AI response and constructs the floating 3D Patient Record cards using the metadata